SmallGPt

March 25, 2026

[1]:
```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset

# ========================
# 1. Load and preprocess text
# ========================

# Load your text file with ~10,000 words
with open("demo.txt", "r", encoding="utf-8") as f:
    text = f.read().lower().replace("\n", " ")

words = text.split()
vocab = sorted(list(set(words)))  # Unique words
vocab_size = len(vocab)
print(f"Vocabulary size: {vocab_size}")

# Create mappings
stoi = {word: i for i, word in enumerate(vocab)}
itos = {i: word for i, word in enumerate(vocab)}

# Parameters
seq_len = 20  # sequence length
embed_size = 64
ff_hidden = 128
num_layers = 4
batch_size = 64

# Prepare sequences
X, Y = [], []
for i in range(len(words) - seq_len):
    seq_in = words[i:i+seq_len]
    seq_out = words[i+1:i+seq_len+1]
    X.append([stoi.get(w, 0) for w in seq_in])
    Y.append([stoi.get(w, 0) for w in seq_out])
```

```
X = torch.tensor(X)
Y = torch.tensor(Y)

dataset = TensorDataset(X, Y)
loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

```
Vocabulary size: 875
```

[2]:
```
# ========================
# 2. Define SmallGPT Model
# ========================

class GPTEmbedding(nn.Module):
    def __init__(self, vocab_size, embed_size, max_len):
        super().__init__()
        self.token_emb = nn.Embedding(vocab_size, embed_size)
        self.pos_emb = nn.Embedding(max_len, embed_size)

    def forward(self, x):
        positions = torch.arange(0, x.size(1), device=x.device).unsqueeze(0)
        return self.token_emb(x) + self.pos_emb(positions)

class SelfAttention(nn.Module):
    def __init__(self, embed_size):
        super().__init__()
        self.query = nn.Linear(embed_size, embed_size)
        self.key = nn.Linear(embed_size, embed_size)
        self.value = nn.Linear(embed_size, embed_size)
        self.scale = embed_size ** 0.5

    def forward(self, x):
        Q = self.query(x)
        K = self.key(x)
        V = self.value(x)
        attn_weights = F.softmax(Q @ K.transpose(-2, -1) / self.scale, dim=-1)
        return attn_weights @ V

class GPTBlock(nn.Module):
    def __init__(self, embed_size, ff_hidden):
        super().__init__()
        self.attn = SelfAttention(embed_size)
        self.ln1 = nn.LayerNorm(embed_size)
        self.ff = nn.Sequential(
            nn.Linear(embed_size, ff_hidden),
            nn.GELU(),
            nn.Linear(ff_hidden, embed_size)
        )
```

```
        self.ln2 = nn.LayerNorm(embed_size)

    def forward(self, x):
        x = x + self.attn(self.ln1(x))
        x = x + self.ff(self.ln2(x))
        return x
```

[3]:
```
class SmallGPT(nn.Module):
    def __init__(self, vocab_size, embed_size, max_len, num_layers, ff_hidden):
        super().__init__()
        self.embedding = GPTEmbedding(vocab_size, embed_size, max_len)
        self.blocks = nn.ModuleList([GPTBlock(embed_size, ff_hidden) for _ in␣
 ↪range(num_layers)])
        self.ln = nn.LayerNorm(embed_size)
        self.head = nn.Linear(embed_size, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        for block in self.blocks:
            x = block(x)
        x = self.ln(x)
        return self.head(x)

model = SmallGPT(vocab_size, embed_size, seq_len, num_layers, ff_hidden)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.CrossEntropyLoss()
```

[4]:
```
# =========================
# 3. Training Loop
# =========================

epochs = 30 # Increase for better learning

for epoch in range(epochs):
    total_loss = 0
    for batch_x, batch_y in loader:
        optimizer.zero_grad()
        logits = model(batch_x)
        loss = loss_fn(logits.view(-1, vocab_size), batch_y.view(-1))
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}/{epochs}, Loss: {total_loss/len(loader):.4f}")
```

```
Epoch 1/30, Loss: 4.9086
Epoch 2/30, Loss: 1.8983
Epoch 3/30, Loss: 0.5583
Epoch 4/30, Loss: 0.2493
```

```
Epoch 5/30, Loss: 0.1674
Epoch 6/30, Loss: 0.1315
Epoch 7/30, Loss: 0.1077
Epoch 8/30, Loss: 0.0887
Epoch 9/30, Loss: 0.0728
Epoch 10/30, Loss: 0.0595
Epoch 11/30, Loss: 0.0484
Epoch 12/30, Loss: 0.0392
Epoch 13/30, Loss: 0.0311
Epoch 14/30, Loss: 0.0246
Epoch 15/30, Loss: 0.0195
Epoch 16/30, Loss: 0.0156
Epoch 17/30, Loss: 0.0122
Epoch 18/30, Loss: 0.0097
Epoch 19/30, Loss: 0.0086
Epoch 20/30, Loss: 0.0076
Epoch 21/30, Loss: 0.0069
Epoch 22/30, Loss: 0.0069
Epoch 23/30, Loss: 0.0074
Epoch 24/30, Loss: 0.0065
Epoch 25/30, Loss: 0.0045
Epoch 26/30, Loss: 0.0039
Epoch 27/30, Loss: 0.0037
Epoch 28/30, Loss: 0.0050
Epoch 29/30, Loss: 0.0088
Epoch 30/30, Loss: 0.0113
```

[10]:
```python
# ========================
# 4. Text Generation
# ========================

def generate_text(model, start_words, length=50):
    model.eval()
    idxs = [stoi.get(word, 0) for word in start_words.split()]
    for _ in range(length):
        x = torch.tensor([idxs[-seq_len:]])
        logits = model(x)
        next_idx = torch.argmax(logits[0, -1]).item()
        idxs.append(next_idx)
    return " ".join([itos[i] for i in idxs])

# Example usage
print(generate_text(model, " watched distant mountains  ",1 ))
```

```
watched distant mountains glowed
```

[14]:
```python
print(generate_text(model, " The cat followed him quietly through the  ",1 ))
```

```
a cat followed him quietly through the table
```

[17]:
```
print(generate_text(model, " The sun warmed their faces as they walked hand ␣
↪",1 ))
```

```
a sun warmed their faces as they walked hand as
```

[21]:
```
print(generate_text(model, "The morning mist covered the fields like a soft",1))
```

```
a morning mist covered the fields like a soft blanket.
```

[25]:
```
print(generate_text(model, "The dog barked at the passing",1))
```

```
a dog barked at the passing cyclist.
```

[ ]:
```

```